

An Investigation of Modularity and Optimization on Robotic Computing Planning Algorithms on FPGAs

Nicole Alexandru
Computer Engineering
Tufts University

Shannon Crandley
Computer Engineering
Tufts University

Lawer Nyako
Computer Science
Tufts University

Abstract—Robotic computing is a prominent technology that allows robots to adapt to different environments and perform tasks, both pre-planned and autonomous. This technology combines traditional robotics with advanced computer science that allows robots to react to dynamic elements. There are different stages within the robotic computing pipeline, including perception, planning, and mapping. This project investigates planning algorithms, as well as optimizes the algorithms to improve efficiency and performance. The algorithms use FPGAs, as there are many benefits to using FPGAs over other ASICs for robotic computing. Lastly, the results are analyzed and compared to the original algorithms.

I. INTRODUCTION

Robotic computing bridges computing technologies with robotic systems. These systems are extremely complex, as they are interdisciplinary and require a full end-to-end implementation, both in hardware and software. The robot must continuously read in sensor values, using these to adapt to the dynamic environment and move fluidly through the robotic computing pipeline. Large amounts of data must be processed in real-time, which requires accurate spatial and temporal synchronization and pre-processing.

The robotic computing pipeline can be difficult to process and meet real-time requirements. Real-time and power constraints of the robotic system hinder its ability to perform well in latency-critical and power-limited situations. Thus, it is very important to choose the proper hardware for the robotic system. Specifically, the ASIC chosen for the robotic computing can drastically affect the performance of the system. CPUs and GPUs are commonly used for commercial compute platforms. Unfortunately, CPUs and GPUs, although beneficial for their general purpose use, consume about 10-100 W of power, which is much higher than what is available on a resource-limited robotic system. On the other hand, FPGAs require low power, and are able to meet real-time requirements while achieving high energy efficiency compared to CPUs and GPUs.

There are many algorithms emerging for robotic computing, but this project investigates the planning stage in the robotic computing pipeline. There are two types of planning algorithms: travel planning and task planning. Simple algorithms of both travel and task planning algorithms are investigated and modularized. Then, a more complex algorithm is investigated and optimized. This algorithm is called the Stochastic Trajectory Optimization for Motion Planning (STOMP) [1].

Noisy trajectories for a robotic arm are added to its initial trajectory in order to optimize its motion trajectory. The STOMP algorithm has been previously developed and used with C code and high level synthesis (HLS) design that converts into FPGA-compatible RTL. This project analyzes and builds off of a previous design developed by researcher Bob (Jubo) Xu at Imperial College London in collaboration with Intel [2]. Various aspects of the design are tested and optimized on a granular level in order to improve performance and power efficiency.

Additionally, the benefits of computing on FPGAs is researched and investigated in relation to these planning algorithms. The planning stage of the pipeline includes the behavior decision and motion planning. The task of motion planning is to generate a trajectory and send it to the feedback control for execution. The simpler algorithms this project investigates are grid-based, such as Dijkstra or A*. Planning algorithms have many tradeoffs with FPGAs compared to other ASICs, which will be outlined in Section II.

A. Related Work

This project is inspired by the book *Robotic Computing on FPGAs (Synthesis Lectures on Computer Architecture)* [3]. This book defines robotic computing and outlines the current advancements of robotic computing on FPGAs. This project investigates the planning algorithms listed within the book and uses the information within the lecture series book to understand the benefits on computing on FPGAs. The data in the book is used to compare the optimized versions of the algorithms to the original algorithms, ensuring the modularity of the FPGA is exploited within this project.

II. MOTIVATION AND GOALS

A. Characteristics of FPGAs

The biggest trait of FPGAs is the ability to continuously reprogram the chip. Modern FPGAs have reconfigurable logic and memory, which allows systems to be continuously updated with power and performance efficiency. FPGAs attract attention from the robotic community because of their energy-efficient platform for robotic computing. FPGAs provide the flexibility of onsite programming and re-programming needing to re-fabricate the system, unlike other ASICs.

In FPGA Architecture, there are four main components: Configurable Logic Blocks (CLBs), Programmable Routing

Blocks (PRBs), I/O Blocks (IOBs), and Digital Signal Processors (DSPs). The CLBs are logic resources within the chip. They link with the PRBs to execute complex logic functions, synchronize code, and implement memory functions. The IOBs bridge signals onto the chip and send them back off again, allowing the FPGA to continuously processor data. The DSPs have been optimized to implement many digital signal processing functions with maximum performance and minimum logic resources used. These components allow for peak energy and performance efficiency.

The flexibility of the FPGA allows partial reconfiguration, which means the FPGA can be partially reconfigured during runtime without jeopardizing the integrity of the parts of the application that are not being reconfigured. Thus, the other part of the FPGA is preserved and continues to operate during runtime. Partial reconfiguration has a large performance overhead due to the configuration. Slow partial reconfiguration limits the program into performing offline hardware updates, as opposed to a fast partial reconfiguration allows swapping physical components during runtime.

B. Benefits of using FPGAs for Robotic Computing Planning Algorithms

The two main components of planning algorithms are real-time and flexibility. Robots are often placed in dynamic environments that require real-time data processing to respond properly to the environments. The volume of data needed to be processed is too large for a CPU to do well in parallel, and a GPU consumes too much power. So, an FPGA is a better choice for the robotic system. Additionally, the flexibility of FPGAs addresses the flexibility needed in robotic systems, as the mapping of the system must be constantly changing with the environment. FPGAs provide a good compromise between high-performance and flexibility.

FPGAs also provide massive parallelism that allows many operations to run simultaneously. For example, candidate trajectories or collision checks need to happen in parallel. Unlike CPUs and GPUs, FPGAs allow predictable latency and deterministic timing. This is ideal for high-frequency replanning in dynamic environments. FPGA data processing has low-latency, so they can achieve a much higher performance per watt, meaning they are more energy efficient than CPUs and GPUs.

Overall, FPGAs provide constant data processing in parallel to create trajectories that are vital for planning algorithms. FPGAs can be very modular, where components are constantly swapping in and out depending on what is needed for the algorithm. The re-programmability of FPGAs is a huge benefit to robotic computing algorithms.

C. Robotic Planning Algorithms

In robotics, after a space is mapped out and turned into a graph of edges and vertices (converting the geographical information of the environment, including the location of the robot and objects to avoid in the environment, known as localization), planning algorithms are used to compute the path or trajectory the robot will take through the environment.

Single-Source Shortest Paths (SSSP) algorithms find the shortest path from a starting location to every other location in a graph. To find the shortest path to a specific point, that would involve using an SSSP algorithm and then only using the resulting path to the wanted point.

In the planning stage, these algorithms are often used to determine the shortest path in the environment. Different algorithms are used depending on the graph implementation and the amount of processing power allocated to planning. A simpler algorithm may require less computational power but could take longer to run. A more complex algorithm could be faster, but due the difficulty of implementing the algorithm efficiently, this can be very computationally intense. When selecting a traversal planning algorithm, the engineers have to be wary of balancing these parameters.

1) *Travel Planning with STOMP*: The STOMP algorithm is a promising example of a motion planning algorithm that can be optimized for use on non-CPU architectures. Heavy use of matrix multiplication, parallel analysis of the robot's three degrees of freedom, and a hardware-based random number generator make it a prime target for optimization using FPGAs. Furthermore, the previous research example of this new design uses a now-discontinued Intel HLS software. The design is functionally unusable, and could be further optimized by writing RTL by hand.

2) *Task Planning*: When there is a system of robots aiming to complete tasks, concurrently with planning out where the robots are, the system needs distribute these tasks between all the robots in the system. Different algorithms prioritize giving tasks to the robot that is most tailored to a specific task, while others prioritize having the tasks be evenly distributed between the robots. During runtime, the system would want to switch which type of algorithm is being used to allocate tasks depending on the current state of the robotic system [4].

III. EXPERIMENTAL DESIGN AND TOOLS

A. Graphical Planning Algorithms

This project implements 3 graph-based planning algorithms, each with increasing complexity: Bellman-Ford, Dijkstra's, and A*. While the experimental design did not include implementing these graphical planning algorithms in an actual robot, this project investigates the differences in power consumption and utilization that come from running these 3 algorithms on an FPGA. With FPGA implementations allowing for increased modularity, this project test if there were enough differences in power usage and utilization for potential modularity to be useful.

These algorithms were written in Verilog, and their power consumption and utilization on a Zynq MPSoC ZCU106 platform were simulated using Vivado 2023.1.

1) *Bellman-Ford*: Bellman-Ford calculates SSSP by checking each edge for improvements on the calculated shortest path. This is done the number of vertices - 1 times (or when the scores for shortest paths stop changing). It builds a tree of the shortest paths in a process called relaxing edges, where if you

can improve the score to a vertex with a given edge, update the path and score for that vertex. The edges are relaxed in an arbitrary order. Unlike the other two algorithms, Bellman-Ford works when the graph includes negative edges (though in robotics, since the traveling graph is made earlier in the robotics pipeline, the graph shouldn't have any negative edges). It is the simplest of these 3 algorithms, but Bellman-Ford's runtime is $\theta(\#Edges * \#Vertices)$, the slowest out of the 3 algorithms implemented.

2) *Dijkstra's*: Dijkstra's algorithm relaxes outward from the source, building a growing tree of shortest paths. To do so, Dijkstra's algorithm makes a priority queue of vertices, each having an associated path length score, popping off the vertex with the minimum score in the priority queue every iteration. It's a greedy algorithm, adding the closest neighboring vertex to the tree of shortest paths each iteration. Its runtime is $\theta((\#Vertices + \#Edges) * \log(\#Vertices))$.

3) *A**: A* is an extension of Dijkstra's algorithm, but it accounts for an estimated distance to an end goal using a heuristic (an algorithm that estimates the minimum cost from any vertex to the goal) to focus the graphical search. Its time complexity depends on how good the heuristic is, being $O(\text{branching factor of heuristic}^{\text{depth of the shortest path}})$ and in the best case, with a perfect heuristic $O(\#Vertices + \#Edges)$.

B. Multi-Robot Task Planning Algorithms

[4] provides pseudocode for 4 algorithms to assist in planning tasks for a multi-robot system. Determining when tasks will be given to the robots (Task Allocation) can be done statically with a round robin algorithm, or dynamically with a priority-based algorithm. Determining which task would be given to which robot is done with either an auction-based algorithm or a load-based algorithm.

The pseudocode was used to write the algorithms in Verilog, and their power consumption and utilization on a Zynq MPSoC ZCU106 platform were simulated using Vivado 2023.1.

1) *Round Robin (Static Task Allocation)*: The round robin algorithm assigns tasks in a set order before the system begins, and the ordering does not change. The robots receive tasks in a cyclic manner.

2) *Priority (Dynamic Task Allocation)*: The priority-based algorithm dynamically assigns the most urgent task first to the most suitable available robot.

3) *Auction (Task Distribution)*: With the auction task distribution algorithm, the robots "bid" for tasks, with tasks being assigned to the most fitting robot. The factors the robots use to decide how much to "bid" are how efficiently they can complete the task, how much energy it would take to do the task, and their given workload.

4) *Load (Task Distribution)*: With the load-based task distribution algorithm, tasks are given based on each robot's current capacity. The goal with this algorithm is to ensure no robot is overloaded while other robots are idle.

C. STOMP (Stochastic Trajectory Optimization for Motion Planning)

Over the course of the project, many robotic kernels, benchmarks, and algorithms are analyzed. Many open-source programs on platforms like GitHub were attempted, but most of the tools are extremely difficult to use or were dead-ends in some way. Attempting to download a variety of tools, packages, and libraries would burn many hours and result in little success. The work this project heavily relies on is a research collaboration between Imperial College London and Intel OneAPI, using Intel's HLS [2]. This has been discontinued for several years, and the authors were unable to download and use the program. Attempts were made to directly translate this code to Vitis HLS format, but this was significantly hindered by lack of previous HLS experience, unique nature of the Vitis syntax, and frequent platform bugs and crashes.

Moving forward, the project primarily uses Mr Xu's reported outputs as the basis for comparison to the project's design. This is not as accurate as collecting data and reproducing their work, but for the sake of time, this is a required shortcut.

This project is also unable to compare its implementation to a purely software-based implementation of STOMP [5]. This project attempted to use several platforms to analyze an example C implementation, all of which had major platform issues. The perf energy package Linux command is unsupported on the Tufts EECS servers, the Intel PCM program failed to run on WSL on our personal computers, and the Sniper simulation is also unable to run the program and generate power data.

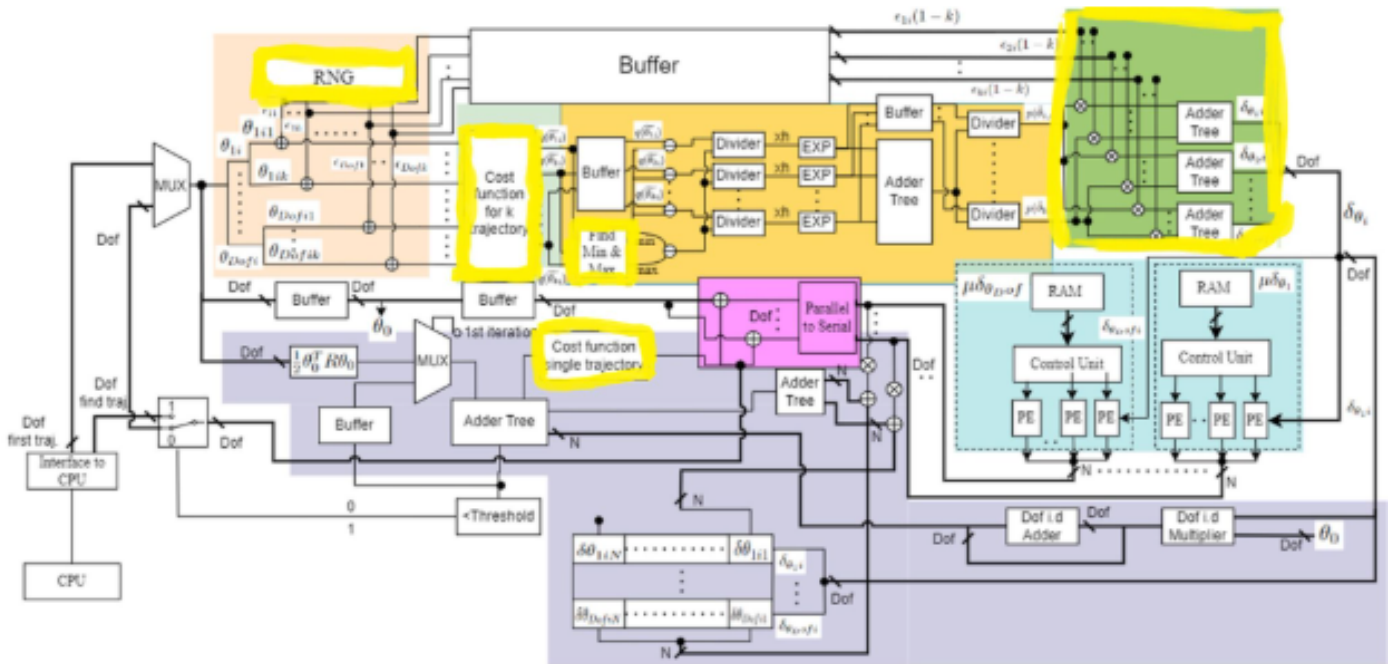
The reference HLS code from Imperial College London is analyzed for potential spots of improvement. A lot of the design used SYCL libraries for HLS synthesis of math functions. Using visual analysis of the code and the author's description of architecture, there are several examples found for improvement for the design.

Figure 1 shows the researcher's diagram of their design. Highlighted in yellow are the blocks with the main architectural changes in the design.

The rest of the design is also implemented in RTL, but is not particularly notable, given that the operations are typical addition, control logic, buffers, etc. The original implementation uses C code with unrolling loops that effectively generate the same RTL as what is hand-written. This project's RTL is designed on the Vivado 2023.1 platform, using a Zynq MPSoC ZCU106 platform as the model for implementation.

The main changes made in the design are to the Random Number Generator (RNG), the max-min function, and mathematical functions like sin, cosine, square root, and exponential.

In the original implementation, a max-min function is written out in semi-RTL style code using Bubble Sort. One by one elements of the array are compared and the maximum and minimum are both calculated in this process. This implementation converts the sorting algorithm into a binary-tree style comparison where all elements are compared at the same time to their neighbor, half are eliminated each clock cycle, and the 'winners' of each comparison are again simultaneously



compared in the next clock cycle. This reduces the algorithmic complexity from $O(N)$ to $O(\log N)$. Another strategy would have been comparing more than two elements together at a time, or even make one massive comparison between all elements in a single clock cycle, but this might have created lengthy critical paths and might have violated the clock period of the system.

Another change to the system was the hardware implementation of math functions. For both sine, cosine, and the exponential (e^x) computations this implementation uses the CORDIC algorithm [6]. This is an algorithm that's built for RTL hardware design. The CORDIC performs rotations around the unit circle many times using just a few hard-coded values for the tangent of certain angles in order to approximate sine and cosine values. It computes things similarly for e^x . Its method of rotation is highly effective because it doesn't use expensive multipliers, but simple shift and add operations [7].

We also used another existing Verilog algorithm for computing the square root computation [8].

The largest change to the design is the implementation of the random number generator in the design. The RNG is first a uniform random number generator which is then tuned to a Gaussian spread. This Gaussian random number is used as a probability distribution for the possible trajectories for a robotic arm. The existing system uses this RTL for the uniform generator [9] and the Hadamard Transform to tune to a Gaussian distribution [10]. This project changes the architecture of both parts, instead using a simple double Linear Feedback Shift Register system suited for FPGAs. The LFSR has an arbitrary bit length (and therefore arbitrary range) based on the range expected for use in the design [11]. This

implementation uses a Xilinx seed number, an LFSR to create a uniform RNG, and a second LFSR as an irregular non-uniform clock to tune the first to a Gaussian. The RTL diagram is shown in Figure 2.

Overall, one of the biggest changes made to the whole design is the conversion from floating point to fixed point calculations. The previous implementation heavily uses floating point numbers and algebra, which is very inefficient and bloated for use on an FPGA. While there are options for floating-point computation and floating-to-fixed IPs available on the Zynq FPGA, this still adds significant power usage and resource utilization. By using algorithms like CORDIC and changing port types, the system can perform math using fixed point calculations and converting to floating point at the very end.

Unfortunately, as the entire system was being developed, modules were created with different types of ports, unpacked vs packed arrays, different bit lengths of inputs, etc. and so there wasn't enough time to correctly stitch together all modules into a final pipelined and clock-aligned design. With more time, all modules would be connected into a top level file and tested on a whole-system basis.

IV. RESULTS

A. Graphical Planning Algorithm Results

As shown in II, Bellman-Ford has the lowest power and utilization out of all of the graphical planning algorithms. This is likely due to Bellman-Ford being the simplest algorithm, particularly since it does not include a priority queue. I shows Dijkstra's with the highest power usage. This is likely due to both its priority queue (making it use more power than

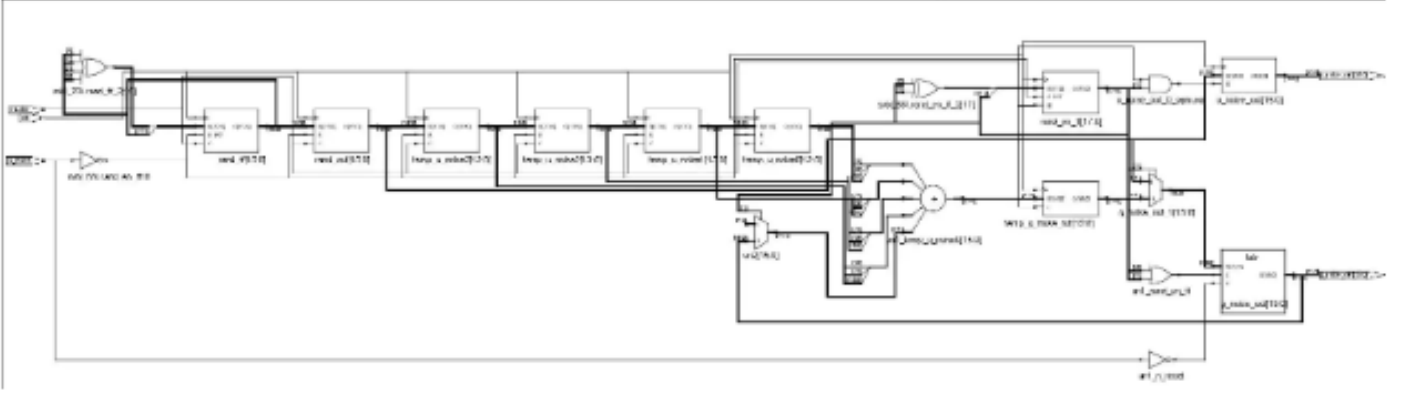


Fig. 2: RTL Diagram of STOMP

TABLE I: Power and Utilization Analysis of Dijkstra's Algorithm

Junction	
Junction Temp	29.1°C
Junction/Ambient Ratio	1.164
Dynamic Power	
Signals	1.602 W
Logic	1.842 W
I/O	0.131 W
Total	3.574 W
Static Power	
	0.611 W
Total Power	
	4.185 W
LUTs	
	328
Registers	
	247

TABLE II: Power and Utilization Analysis of Bellman-Ford Algorithm

Junction	
Junction Temp	26.8°C
Junction/Ambient Ratio	1.072
Dynamic Power	
Signals	0.536 W
Logic	0.419 W
I/O	0.241 W
Total	1.196 W
Static Power	
	0.598 W
Total Power	
	1.794 W
LUTs	
	70
Registers	
	75

Bellman-Ford) and to exploring directions that are not the shortest path. While the experiment did not show the time complexity improvement of A* compared to Dijkstra's, III shows that A* had a lower power usage versus Dijkstra's, possibly an indication of its increased efficiency. A* also has the most LUTs, likely due to its increased complexity (due to the heuristic). Lastly, A* and Dijkstra's have the same number of registers because A* is an extension of Dijkstra's.

B. Task Planning Algorithm Results

IV and V show that the utilization and power for the round robin task allocation algorithm are slightly more than those for the priority-based algorithm. This indicates that choosing between which of the two to run should be based on situational context. Similarly, as shown in VI and VII, the utilization and power for the auction task distribution algorithm are slightly more than those for the priority-based algorithm, indicating that selecting between these two algorithms should also be a

contextual decision. Comparing [4]'s implementation VIII to IV, V, VI, and VII, this project's implementation of the paper's given pseudocode has a much lower utilization. This is likely because this project's implementation does not include logic for switching between algorithms and connecting to other modules, and [4]'s results combine the utilization of both task allocation algorithms and both task distribution algorithms.

C. STOMP Optimization Results

The changes that this implementation made to the RTL are effective both in performance and power efficiency. Most modules use very few LUTs upon implementation. One instance of the CORDIC algorithm is shown in Figures 3 and 4, as well as Table IX. 1007 LUTS (0.4% of total available) are used and 989 registers are used (0.2% of total available). Predicted power usage is large, at 71.142 W, however, when estimating usage the project disregards I/O power. This is because the module is unattached from top level modules that use different

TABLE III: Power and Utilization Analysis of A* Algorithm

Junction	
Junction Temp	26.3°C
Junction/Ambient Ratio	1.132
Dynamic Power	
Signals	1.178 W
Logic	1.470 W
I/O	0.080 W
Total	2.728 W
Static Power	
0.606 W	
Total Power	
3.335 W	
LUTs	
338	
Registers	
247	

TABLE IV: Power and Utilization Analysis of Round Robin (Static Task Allocation) Algorithm

Junction	
Junction Temp	27.7°C
Junction/Ambient Ratio	1.108
Dynamic Power	
Signals	0.009 W
Logic	0.020 W
I/O	2.079 W
Total	2.108 W
Static Power	
0.603 W	
Total Power	
2.711 W	
LUTs	
1	
Registers	
2	

input and output ports and will likely be connected to the Zynq processor using an AXI bus instead of pure I/O ports, which consume much more power. Therefore, the project considers this design to be using 20.7 W.

Because this implementation uses fixed-point computation and these simple algorithms, there is a loss of resolution and accuracy. For example, when using CORDIC for the exponential, there is a strict convergence range for x , $(-1.12, 1.12)$. Additional modifications are needed to handle overflow, decomposing x into multiple components to increase the range. Overall, the CORDIC algorithm can be tuned to increase accuracy (while reducing performance). The CORDIC algorithm runs in an iterative fashion to get closer and closer to the true answer. With each iteration, one bit of resolution is added.

TABLE V: Power and Utilization Analysis of Priority-Based (Dynamic Task Allocation) Algorithm

Junction	
Junction Temp	26.0°C
Junction/Ambient Ratio	1.104
Dynamic Power	
Signals	0.005 W
Logic	0.004 W
I/O	0.440 W
Total	0.449 W
Static Power	
0.594 W	
Total Power	
1.043 W	
LUTs	
1	
Registers	
0	

TABLE VI: Power and Utilization Analysis of Auction-Based (Task Distribution) Algorithm

Junction	
Junction Temp	26.3°C
Junction/Ambient Ratio	1.052
Dynamic Power	
Signals	0.114 W
Logic	0.094 W
I/O	0.525 W
Total	0.733 W
Static Power	
0.596 W	
Total Power	
1.328 W	
LUTs	
17	
Registers	
0	

Depending on the accuracy needs of the robot itself, this can be tuned to optimize for speed or accuracy.

Another loss in accuracy is with the square root module- it generates a whole number value as the square root and a remainder as well. This design only uses the whole number and disregards the remainder, but additional overhead could be added to include it.

One of the most consequential improvements to the system was the change in RNG. This implementation is able to run an implementation of the uniform RNG component of the original design on an FPGA, and uses reported data for the Gaussian transformation. The original URNG used 3.447 W and 64 LUTs. The authors don't have power data for the

TABLE VII: Power and Utilization Analysis of Load-Based (Task Distribution) Algorithm

Junction	
Junction Temp	26.2°C
Junction/Ambient Ratio	1.048
Dynamic Power	
Signals	0.033 W
Logic	0.019 W
I/O	0.575 W
Total	0.628 W
Static Power	
0.595 W	
Total Power	
1.223 W	
LUTs	
6	
Registers	
0	

TABLE VIII: Task Allocation and Distribution Modules from [4]

Task Allocation Module	Task Distribution Module
95	98
18	20
16	18

Gaussian transform, but know the design used between 102-204 LUTs. The URNG used 1.339 W and 11 LUTs, and the Gaussian transform used 33 LUTs and 1.35 W. This amounts to a 61% power reduction for the URNG component, and a 74-84% reduction in total LUTs. Using LUTs as a proxy for power, the authors can also estimate that our total power usage was less than the originals.

The project also tested this system for accuracy. The system is very close to a perfect Gaussian. After collecting 1000 samples, D'Agostino-Pearson is performed to test the design.

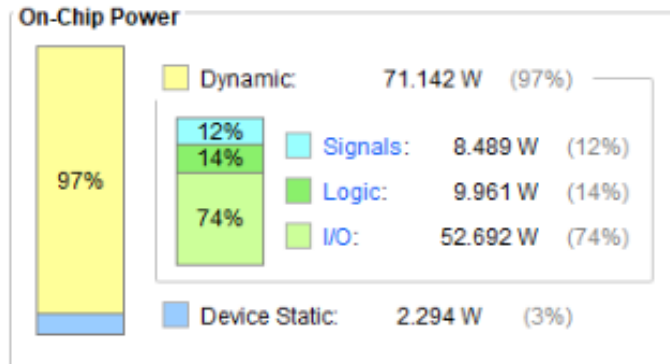


Fig. 3: CORDIC Power Output

TABLE IX: Power Analysis of STOMP

Dynamic Power	
Signals	8.489 W
Logic	9.961 W
I/O	52.692 W
Total	71.142 W
Static Power	
2.294 W	
Total Power	
73.436 W	

This normality test computes how skewed the shape of the distribution is from a Gaussian distribution. The system has a p value of 0.005, showing that the system is very close to a true Gaussian. Figure 5 shows the frequency distribution of these values when using a 32-bit range.

While the project wasn't able to collect data from the original implementation, a provided graph (Figure 6) shows the shape of their distribution. Just using visual inspection, this distribution does not look very Gaussian, with a non-smooth center and tails [2]. In this project, the authors feel very confident in the accuracy of the design and its power optimization.

V. ANALYSIS OF RESULTS

A. Analysis of STOMP Optimization

Overall, it seems like the optimizations of the algorithm and could perform very quickly and efficiently on FPGAs. This project used a variety of open-source research and algorithms to effectively run this algorithm using fixed-point math and parallel operations. Despite many challenges related to libraries and software, a significant amount of RTL code has been optimized.

With more time and resources, this project would've combined all the components together, attempted to run placement optimizations using Vivado, and compared power and performance to CPUs, HLS implementations, and even ASIC implementations.

B. Benefits of Modularity

Modularity is a large component of many robotic computing algorithms, as it directly affects how efficiently the system can map complex algorithms onto the hardware, reuse resources, and manage power and performance tradeoffs. The largest benefit of modularity found in this project was the power reduction due to resource sharing, both in STOMP and the simpler planning algorithms. Both modularity and optimization allowed for power efficiency and performance efficiency. By utilizing parallelization and reducing hardware redundancy, this implementation of STOMP and the other planning algorithms test the limits of computing on FPGAs. With the task-focused planning algorithms, the modularity of FPGAs can be used to swap out which algorithm is being utilized on hardware.

Name	CLB LUTs (230400)	CLB Registers (460800)	CARRY8 (28800)	CLB (28800)	LUT as Logic (230400)	Bonded IOB (350)	HPIOB_M (144)	HPIOB_S (144)	HIOB_M (24)	HIOB_S (24)	GLOBAL CLOCK BUFFERS (544)
N CORDIC	1007	989	144	170	1007	97	40	40	9	8	1

Fig. 4: CORDIC Utilization

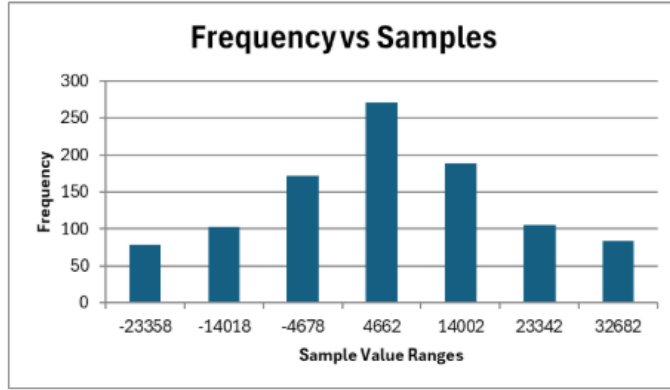


Fig. 5: Frequency Distribution of Accuracy Test

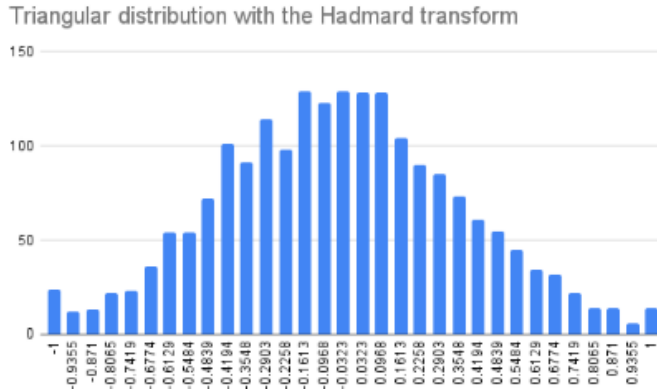


Fig. 6: Hadamard Transform Frequency Distribution

Since both task distribution and task allocation algorithms have similar power usage and utilization, the algorithm selected would be the most conducive for a given situation. For STOMP and the simpler graphical planning algorithms, the decision is about balancing an increase in power and utilization with an improvement in speed. As these planning algorithms are a larger part of a whole robotics pipeline, if a robot is being run on a limited FPGA, it may be worth it to swap which algorithm is being run to reduce power usage. With STOMP, a robot can swap between the specific distributions of [1] implementation or this project's implementation. With the simpler planning algorithms, if the robot has the bandwidth to utilize A* or Dijkstra's algorithm, it can do so, but in moments when power or utilization is limited, the robot can switch to using Bellman-Ford, sacrificing overall runtime.

REFERENCES

[1] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in

Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Shanghai, China, 2011, pp. 4569–4574, doi: 10.1109/ICRA.2011.5980280. [Accessed: May 3, 2026].

[2] Jubo-Xu, "FPGA-Implementation-of-STOMP-in-Robotics," GitHub, 2025. [Online]. Available: <https://github.com/Jubo-Xu/FPGA-Implementation-of-STOMP-in-Robotics>. [Accessed: May 3, 2026].

[3] R. C. Luo, *Robotic Computing on FPGAs*, Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers. [Accessed: March 10, 2026].

[4] M. Basha, G. D. Vani, M. C. Chinnaiiah, M. Khaleel, N. Bhasakar, and M. Ganesh, "Design and Implementation of Task Allocation and Task Distribution Methods for Multi-Robotic System Using FPGA," in *Proc. 2025 IEEE 5th Int. Conf. VLSI Systems, Architecture, Technology and Applications (VLSI SATA)*, Bangalore, India, 2025, pp. 1–6, doi: 10.1109/VLSISATA65374.2025.11069976. [Accessed: May 3, 2026].

[5] ros-industrial, "stomp: Stochastic Trajectory Optimization for Motion Planning (STOMP)," GitHub, 2025. [Online]. Available: <https://github.com/ros-industrial/stomp>. [Accessed: May 3, 2026].

[6] D. B. Thomas, "FPGA Gaussian Random Number Generators with Guaranteed Statistical Accuracy," in *Proc. IEEE 22nd Annual Int. Symp. Field-Programmable Custom Computing Machines (FCCM)*, Boston, MA, USA, 2014, pp. 149–156, doi: 10.1109/FCCM.2014.47. [Accessed: May 3, 2026].

[7] cebarnes, "cordic/cordic.v," GitHub, 2025. [Online]. Available: <https://github.com/cebarnes/cordic/blob/master/cordic.v>. [Accessed: May 3, 2026].

[8] "An Introduction to the CORDIC Algorithm," All About Circuits. [Online]. Available: <https://www.allaboutcircuits.com/technical-articles/an-introduction-to-the-cordic-algorithm/>. [Accessed: May 3, 2026].

[9] projf, "projf-explore/lib/maths," GitHub, 2025. [Online]. Available: <https://github.com/projf/projf-explore/tree/main/lib/maths>. [Accessed: May 3, 2026].

[10] D. Thomas and W. Luk, "FPGA-Optimised Uniform Random Number Generators using LUTs and Shift Registers." [Online]. Available: https://cas.ee.ic.ac.uk/people/dt10/research/thomas-10-lut_sr_rngs.pdf. [Accessed: May 3, 2026].

[11] DigiKey Electronics, "Pseudo Random Number Generator with Linear Feedback Shift Registers (Verilog)," DigiKey TechForum, Sep. 19, 2024. [Online]. Available: <https://forum.digikey.com/t/pseudo-random-number-generator-with-linear-feedback-shift-registers-verilog/13440>. [Accessed: May 3, 2026].